



Bases de datos

A diferencia de las preferencias compartidas, las bases de datos controlan una gran cantidad de datos estructurados.

SQLite es un motor de bases de datos muy popular actualmente ya que ofrece características muy interesantes como: tamaño pequeño, no necesita servidor, precisa poca configuración, ser transaccional (se garantiza la consistencia, atomicidad, aislamiento y durabilidad, ACID) y es de código libre.

La biblioteca de persistencias Room brinda una capa de abstracción para SQLite que permite acceder a la base de datos sin problemas y, al mismo tiempo, aprovechar toda la tecnología de SQLite. En particular, Room brinda los siguientes beneficios:

- Verificación del tiempo de compilación de las consultas en SQL
- Anotaciones de conveniencia que minimizan el código estándar repetitivo y propenso a errores
- Rutas de migración de bases de datos optimizadas

VERSIÓN ACTUAL DE ROOM 2.8.3

Lo primero que debemos hacer para poder utilizar la librería Room dentro de nuestro proyecto, es indicar las dependencias dentro del fichero build.gradle.

```
dependencies {  
  
    implementation ("androidx.room:room-runtime:2.8.3")  
  
    ksp ("androidx.room:room-compiler:2.8.3")  
  
    implementation ("androidx.room:room-ktx:2.8.3")  
}
```

KSP

Agrega el complemento KSP al proyecto

```
id("com.google.devtools.ksp") version "2.0.21-1.0.27" apply false
```

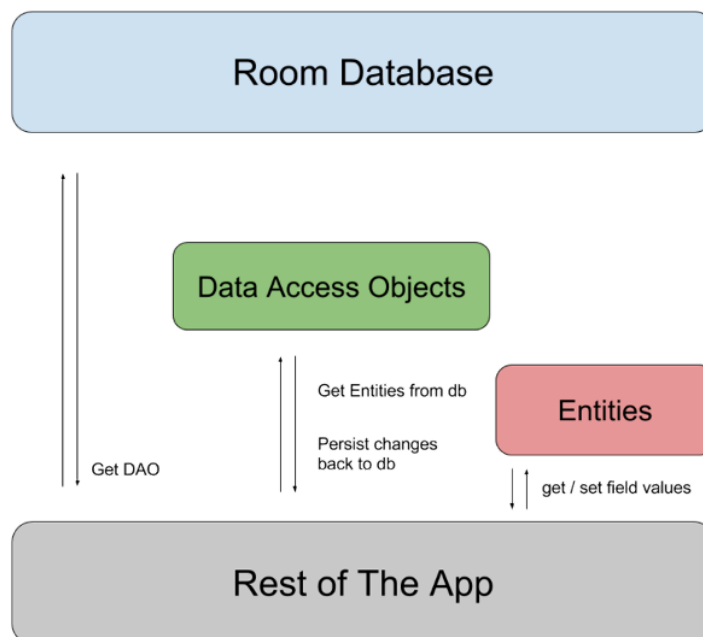
Luego, habilita KSP en el archivo build.gradle.kts a nivel del módulo:



```
id("com.google.devtools.ksp")
```

Gracias a que room introduce una capa de abstracción, las tareas de configuración, creación y trabajo sobre una base de datos local se reducen bastante. Pero antes de empezar a trabajar, es necesario entender muy bien cada uno de los objetos que forman parte de dicha configuración.

- Entity: representa cada una de las tablas o entidades que se guardarán dentro de la base de datos. Están representadas por data class
- DAO: representa cada una de las acciones que se pueden realizar sobre la base de datos. Son las queries que se aplicarán sobre las tablas. Están representadas por interfaces
- RoomDatabase: La clase principal que engloba las entities y DAO los cuales conforman la base de datos.



Entityts

Las entityts representan las tablas que formarán parte de la base de datos. Para poder crear una entity es necesario crear una clase con el decorador `@Entity`, donde en el constructor se indicará cada uno de los elementos que formarán parte de la tabla. Para poder definir estos elementos cabe destacar los siguientes configuradores:



- **@Entity**: Marca una clase para ser mapeada por Room como tabla. El nombre por defecto de la tabla será el de la clase. Para especificar uno distinto usa la propiedad `tableName`.
- **@PrimaryKey**: define la propiedad como clave principal. Es posible poner la anotación `autoGenerate` (boolean) para indicar que la clave sea generada automáticamente y por lo tanto autoincremental
- **@ColumnInfo**: Permite la personalización de la columna asociada con este campo. El atributo `name` permite cambiar el nombre de la columna (por defecto es el nombre del campo)
- **@Ignore**: indica que la propiedad sobre la que se configura no será tomada en cuenta en la base de datos, y lo tanto no la guardará.
- **@ForeignKey**: Indica que el campo donde se configure tendrá una relación con la clave primaria de otra tabla
- **@NonNull**: Denota que un campo, parámetro o valor de retorno de un método no puede ser null. Con ella Room añade la restricción NOT NULL a la columna.

Nota: Los nombres de tablas y columnas en SQLite no distinguen mayúsculas de minúsculas.

En el caso de querer marcar la primary key como autoincremental, es necesario marcarla fuera del constructor

```
@Entity(tableName = "users")
data class Usuario(
    @ColumnInfo(name = "name") var nombre:String,
    @ColumnInfo(name = "email") var correo: String
) {
    @PrimaryKey(autoGenerate = true) var id: Int =0;
}
```



DAO

Un DAO representa un mapeado de operaciones SQL a métodos.

En este caso, los DAO representan cada una de las acciones que se pueden hacer sobre las tablas de la base de datos. Lo bueno que tiene este tipo de objetos (representados mediante interfaces) es que en muchos de los casos no es necesario escribir la sentencia, ya que la librería room se encarga de interpretarla.

Como ya se ha dicho, para poder crear un DAO es necesario crear una interfaz con el decorador `@DAO` y tantos métodos como se consideren necesarios, donde cada uno de ellos tiene el decorador `@Query` con la sentencia que debe ejecutar, a excepción del `@insert` `@delete` y `¿¿@update??`.

En aquellas funciones donde no se define una query (como en el `@insert` o `@delete`), las operaciones se realizan sobre el id o primary key del objeto.

Un ejemplo de DAO sería el siguiente

```
@Dao

interface UsuarioDAO{

    @Query("Select * from users")

    fun selectAll(): List<Usuario>

    @Query("Select * from users Where name = :name")

    fun getByName(name: String): List<Usuario>

    @Query("Delete from users Where name = :name")

    fun deleteByName(name: String)

    @Query("SELECT * FROM users WHERE email = :correo LIMIT 1")

    fun getByEmail(correo: String): Usuario?
```



```
@Query("Update users set email= :newMail WHERE email= :oldMail")

fun updateByEmail(oldMail: String, newMail: String)

@Insert

fun insert(usuario: Usuario): Long

@Delete

fun delete(usuario: Usuario)

}
```

Database

Es el punto de entrada principal para comunicar el resto de tu app con el esquema relacional de datos.

Una vez se han creado tanto las entidades como las operaciones que se pueden hacer sobre ella, el siguiente paso es crear la base de datos. Para ello es necesario crear una clase abstracta que herede de RoomDatabase sobre la cual se aplica el decorador @Database, indicando las entidades que la forman, el número de versión que tiene la base de datos y los dao que se aplicarán sobre la base de datos. Un ejemplo con la clase usuario creada anteriormente sería:

```
@Database(entities = [Usuario::class], version = 1, exportSchema = false)
abstract class UsuariosBBDD: RoomDatabase(){
    abstract fun usuarioDAO(): UsuarioDAO
}
```



Como usar la BBDD

La base de datos aún no se ha creado, ya que lo hará en el primer momento en el que se llame al método build que se va a explicar ahora.

```
@Database(entities = [Usuario::class], version = 1, exportSchema = false)
abstract class UsuariosBBDD: RoomDatabase(){
    abstract fun usuarioDAO(): UsuarioDAO

    companion object DatabaseBuilder{
        private var INSTANCE : UsuariosBBDD ? = null
        fun getInstance (context: Context): UsuariosBBDD {
            if (INSTANCE == null) synchronized(this) {
                INSTANCE = buildRoomDB(context)
            }
            return INSTANCE!!
        }

        private fun buildRoomDB (contexto : Context) =
            Room.databaseBuilder (
                contexto.applicationContext,
                UsuariosBBDD::class.java, "usuarios.db"
            ).build ()
    }
}
```

Para poder crear la base de datos, se utiliza un método companion. Un companion object es un bloque de código que puede contener propiedades y métodos estáticos.

En este caso, se está utilizando un companion object dentro de la clase UsuariosBBDD para manejar la instancia única de la base de datos (patrón singleton) llamado databaseBuilder.

Este método tiene como parámetros el contexto, la clase que representa la base de datos y el nombre de la misma. En el caso de no existir la crea en la ruta local, y en el caso de encontrarla no la crea, simplemente la captura para poder utilizarla. Por último se ejecuta el método build para crear la base de datos.

La función getInstance, en el caso de ser esta variable null la creará mediante la ejecución del método databaseBuilder(). En el caso de ser diferente de null (porque ya se haya llamado en algún momento), se devolverá la propia variable.



- @Database: Marca la clase como una base de datos Room. Usamos las propiedades:
- entities: La lista de entidades incluidas en la base de datos
- version: Versión de la base de datos
- exportSchema: Le dice a Room que exporte el esquema

Una vez creado este patrón, para poder obtener una instancia de la base de datos basta con llamar al método getInstance(), el cual devolverá una instancia nueva de la BD o una ya existente

```
val database = UsuariosBBDD.getInstance(this)
```

Con todo esto, la variable database tiene el acceso a todos los métodos del dato a partir de las funciones que se han declarado.

```
database.usuarioDAO().insert(Usuario("Daniel", "daniel.amiguet@murciaeduca.es"))
```

En android, y en especial en kotlin hay ejecuciones que no deben hacerse en cualquier sitio ya que de hacerse sin control pueden saturar el hilo principal de ejecución. Las bases de datos y sus consultas son un ejemplo típico de esto.

```
lifecycleScope.launch(Dispatchers.IO) {  
    // acción a ejecutar en segundo plano  
    val database = UsuariosBBDD.getInstance(applicationContext)  
    database.usuarioDAO().insert(Usuario("Daniel",  
    "daniel.amiguet@murciaeduca.es"))  
}
```

Para consultar nuestra bases de datos debemos de acceder en nuestro Android Studio, a View > Tool Windows > App Inspection en la barra de menú.



Ejemplo 2:

```
val usuario =  
database.usuarioDAO().getByEmail("daniel.amiguet@murciaeduca.es")  
if(usuario!=null){  
    database.usuarioDAO().delete(usuario)  
}
```